

Reviewer Pack — Minimal Root Separation of Tropical Curves and Resolution Pr...

Entry 065246f51ff9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 17:27:16 UTC

Minimal Root Separation of Tropical Curves and Resolution Proof Length

Entry ID: 065246f51ff9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 17:27:16 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

[‘For a given Tseitin formula, the length of its shortest resolution proof is at least $2^{(\min_{i,j} (t_i - t_j))}$ where $t_1, \dots, t_n$ are the tropical coordinate values of the variables on the curve defined by the formula.’, ‘The ratio between the resolution proof length and the minimum root separation is upper bounded by a polynomial in the number of variables.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a rich framework for studying arithmetic properties of discrete structures. By associating tropical curves to Tseitin formulas, we can leverage the tools of tropical geometry to analyze resolution proof complexity.’, ‘The minimum root separation on a tropical curve could potentially serve as an algebraic lower bound for the length of resolution proofs, providing new insights into the complexity of satisfiability problems.’]

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ab38e73a21b2bcc6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 95% of the seeds tested, the ratio of the shortest resolution proof length to the minimal root separation is within a polynomial bound as defined by the experimental setup.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Tropical Geometry" AND "Resolution Proof Complexity" - "Minimal Root Separation" IN Tropical Geometry AND Resolution Proof Length - "Tseitin formula" AND resolution proof length AND tropical coordinate values

Top relevant hits considered: - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2402.00542v1] Hardness of Random Reordered Encodings of Parity for Resolution and CDCL - [http://arxiv.org/abs/2006.04124v1] On the Complexity of Branching Proofs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_tseitin_formula(n):
    variables = [f'x{i+1}' for i in range(n)]
    clauses = []

    # Generate clauses for each literal and its negation
    for i in range(n):
        clauses.append(f'{variables[i]} v {variables[n+i]}')
        clauses.append(f'¬{variables[i]} v ¬{variables[n+i]}')

    # Generate clauses for the Tseitin formula
    t = n * 2
    for i in range(1, n):
        clauses.append(f'({variables[0]} v x{i+1}) -> {t}')
        t += 1

    return variables, clauses

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented_matrix = [A[i] + [b[i]] for i in range(m)]
```

```

for col in range(n):
    pivot_row = -1
    for row in range(col, m):
        if augmented_matrix[row][col] != 0:
            pivot_row = row
            break

    if pivot_row == -1:
        continue

    # Swap rows to make the pivot element 1
    augmented_matrix[col], augmented_matrix[pivot_row] = augmented_matrix[pivot_row], augmented_matrix[col]

    # Normalize the pivot row
    for j in range(n + 1):
        augmented_matrix[col][j] /= augmented_matrix[col][col]

    # Eliminate other rows
    for row in range(m):
        if row != col:
            factor = augmented_matrix[row][col]
            for j in range(n + 1):
                augmented_matrix[row][j] -= factor * augmented_matrix[col][j]

    return [row[-1] for row in augmented_matrix]

def shortest_resolution_proof_length(clauses):
    n = len(clauses)
    A = [[0] * (n + 1) for _ in range(n)]
    b = [0] * n

    for i in range(n):
        clause = clauses[i]
        if 'v' not in clause:
            continue

        literals = clause.split(' v ')
        for literal in literals:
            if literal.startswith('-'):
                j = int(literal[1:]) - 1
                A[i][j] = -1
            else:
                j = int(literal) - 1
                A[i][j] = 1

        b[i] = 1

    return len(gaussian_elimination(A, b))

def minimal_root_separation(tropical_coordinates):
    min_sep = float('inf')
    for i in range(len(tropical_coordinates)):
        for j in range(i + 1, len(tropical_coordinates)):
            sep = abs(tropical_coordinates[i] - tropical_coordinates[j])
            if sep < min_sep:
                min_sep = sep

```

```

    return min_sep

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_length = 0
    total_count = 0

    for n in n_values:
        variables, clauses = generate_tseitin_formula(n)
        resolution_length = shortest_resolution_proof_length(clauses)

        # Simulate tropical coordinates (for simplicity, use random values)
        tropical_coordinates = [random.uniform(0, n) for _ in range(n)]
        min_sep = minimal_root_separation(tropical_coordinates)

        total_length += resolution_length
        total_count += 1

    avg_length = total_length / total_count
    conjecture_holds = avg_length >= 2 ** (min_sep - 1)

    return {
        "metric_name": "Average Resolution Proof Length",
        "metric_value": avg_length,
        "instances_tested": total_count,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"min_sep={min_sep}, avg_length={avg_length}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    avg_length = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={avg_length} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={avg_length} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"min_sep too large\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aa71739f.py", line 132, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aa71739f.py", line 105, in run_trial
    variables, clauses = generate_tseitin_formula(n)
                        ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aa71739f.py", line 24, in generate_tseitin
    clauses.append(f'{variables[i]} v {variables[n+i]}')
                  ~~~~~^~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the conjecture could not be supported or falsified based on the available results. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13697
2	preregistration	ollama_remote	glm4:latest	0	0	8972
3	novelty	ollama_remote	glm4:latest	0	0	8207
4	novelty	ollama_remote	glm4:latest	0	0	9926
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13594
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13872
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12130
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13667
9	judge	ollama_remote	glm4:latest	0	0	15058

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109123 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/065246f51ff9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/065246f51ff9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/065246f51ff9.tar.gz` (if generated)